
Unstoppable Fabric: Deep Dive into Scalable, AI-Ready Spark in Microsoft Fabric





Unstoppable Fabric:

Deep Dive into Scalable, AI-Ready Spark in Microsoft Fabric

Iurii (Yurri) Iurchenko

Sr Data Engineer, Azure Advocate, Utah Fabric Group Co-lead

Agenda

In this session, we'll take a practical deep dive into Spark-based data engineering in Microsoft Fabric. We'll walk through proven techniques and repeatable patterns for solving common and advanced data engineering challenges, covering advanced Spark SQL functions, effective approaches to semi-structured data, and strategies for building reliable, maintainable pipelines. We will be focusing on Fabric-specific aspects of Spark Notebooks, including table optimization.

AI-related capabilities in Fabric Spark will also be covered, including built-in and integrated functions that support text processing, enrichment, classification, and feature engineering directly within Spark workloads, along with guidance on where these functions fit.

Audience: The presentation is technical and aimed at data engineers and analytics engineers using Spark in Fabric. The session is beginner-friendly, with roughly 50% focused on advanced techniques.

Agenda

01 What is Spark's Mission in Fabric?

Medallion, PySpark, Spark SQL, Python Notebooks, One Lake, Alternative Power Query, Data Factory, & AI

02 Contexts for Spark Usage

Healthcare Data, API data, RealTime Data & NRT, Operational Automation, Complex Logic, Data Cleaning

03 Technical Level in Spark

Idempotence (overwriting, partitioning, merging), API (explode, text, jsonl), Custom libraries (UDF) + its performance, Multithreading read, Delta tables optimization, dedup & cleaning, performance

04 AI & Spark – From Good to Great

Fundamentals (fuzziness, categories, conclusion, additional info), Custom AI functions, Spark & ML

05 Fabric's Spark

Modularity (two versions), Environments, Key Vault, Capacity Consumption & high concurrency

06 Spark – Decision Making & Conclusions

Open floor and action items

01

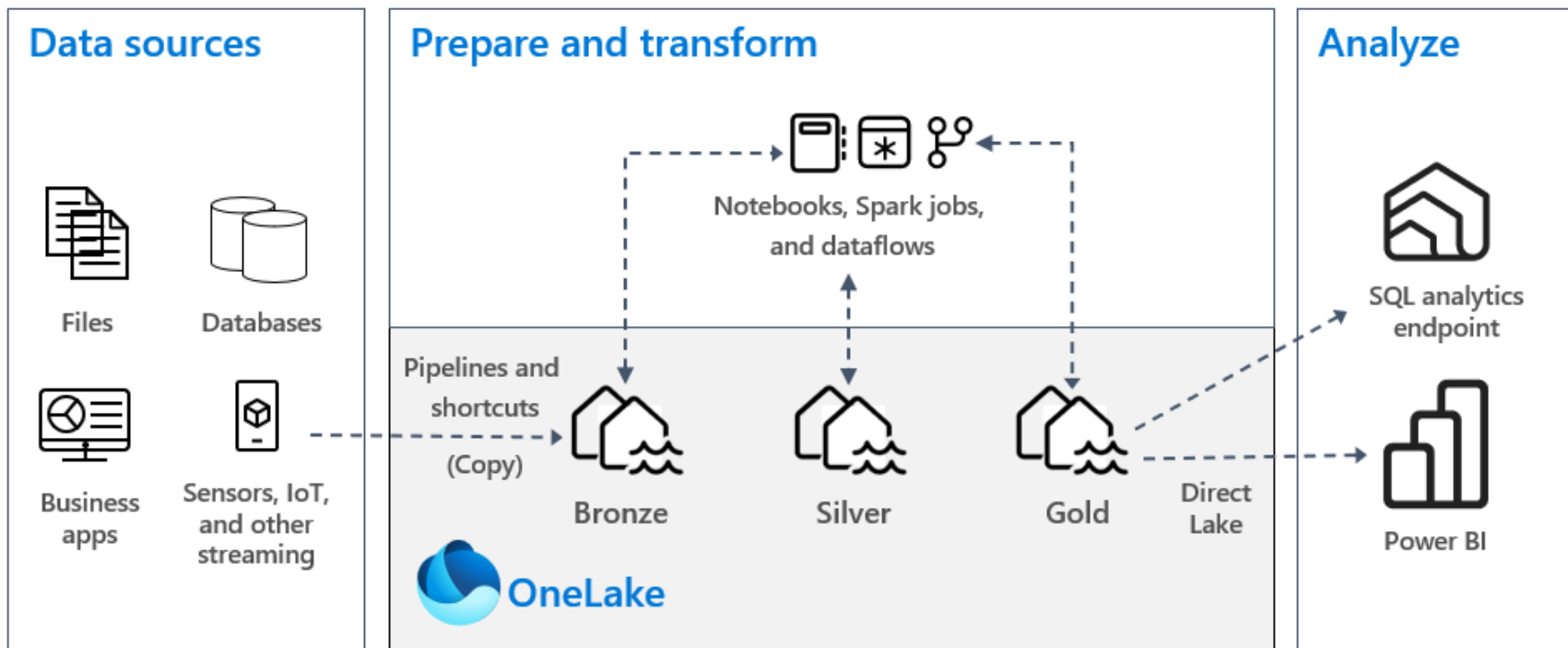


What is Spark's Mission in Fabric?

IN THIS SECTION

- ▶ Medallion architecture
- ▶ PySpark, Spark SQL, and Streaming Spark
- ▶ Spark & One Lake
- ▶ Data pipelines, Power Queries, and Spark – what and when

Medallion Architecture



PySpark vs Spark SQL

Use PySpark:

- Folders
- Loops
- Read / Write

Use Spark SQL:

- Business Logic
- Transformations
- Aggregations

```
1 data = [("Alice", 30), ("Bob", 25), ("Charlie", 35)]
2 df = spark.createDataFrame(data, ["name", "age"])
3 display(df)
```

✓ 6 sec - Command executed in 3 sec 184 ms by Iurii Iurchenko on 3/13/2026, 6:20:06 PM

```
1 %%sql
2 SELECT 'Alice' AS name, 30 AS age
3 UNION ALL
4 SELECT 'Bob', 25
5 UNION ALL
6 SELECT 'Charlie', 35
```

[1] ✓ 9 sec - Session ready in 6 sec 915 ms. Command executed in 2 sec 700 ms by Iurii Iurchenko

>  Spark jobs (1 of 1 succeeded)  Resources

 Table

 New chart

Table view

	ABC name	123 age	
1	Alice	30	
2	Bob	25	
3	Charlie	35	

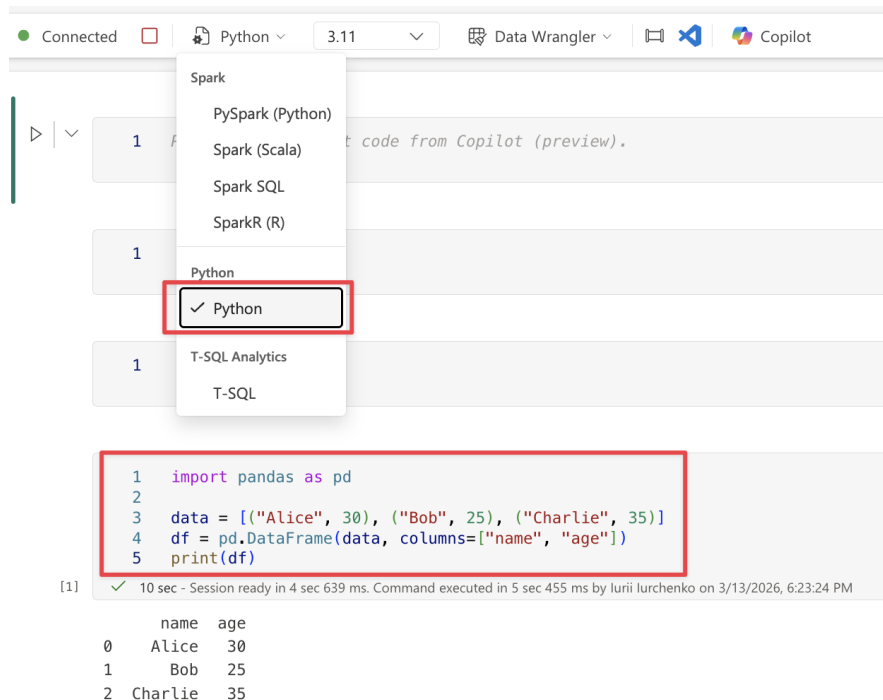
Python Notebooks vs Spark Notebooks

Spark Notebooks:

- Medium-to-large datasets
- PySpark, Spark SQL, Delta tables
- 8 CU consumption

Python Notebooks:

- Small-to-medium datasets
- Pandas, scikit-learn, matplotlib
- 2 CU consumption



The screenshot shows the Databricks notebook interface. At the top, there's a toolbar with 'Connected', 'Python' (selected), '3.11', 'Data Wrangler', and 'Copilot'. A dropdown menu is open, showing options: Spark, PySpark (Python), Spark (Scala), Spark SQL, SparkR (R), Python (selected with a checkmark), T-SQL Analytics, and T-SQL. Below the menu, a code cell is visible with the following Python code:

```
1 import pandas as pd
2
3 data = [("Alice", 30), ("Bob", 25), ("Charlie", 35)]
4 df = pd.DataFrame(data, columns=["name", "age"])
5 print(df)
```

The code cell is highlighted with a red box. Below the code, the execution status is shown: [1] ✓ 10 sec - Session ready in 4 sec 639 ms. Command executed in 5 sec 455 ms by Iurii Iurchenko on 3/13/2026, 6:23:24 PM. The output of the code is a table:

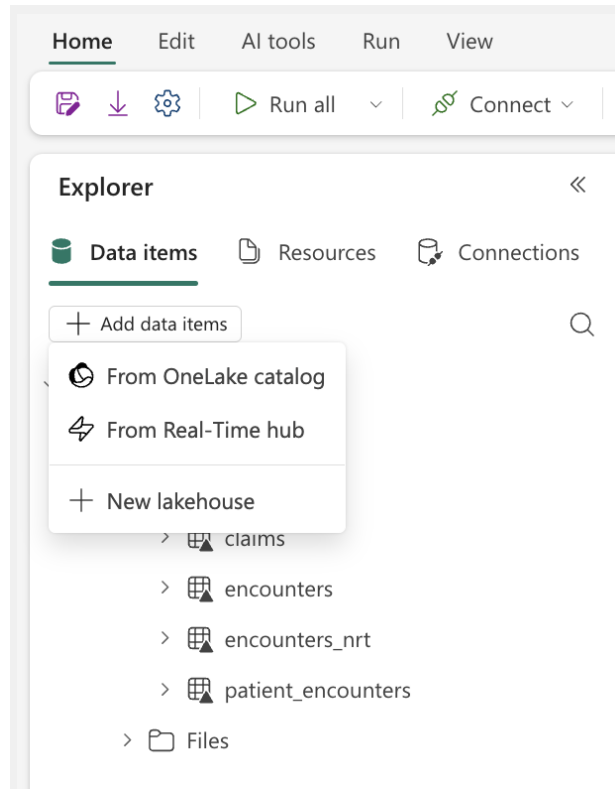
	name	age
0	Alice	30
1	Bob	25
2	Charlie	35

Spark vs Power Query vs Data Pipelines

	Spark	Power Query	Data Pipeline
User Profile	Engineer	Business User	Engineer
Data Volume	Any	Small-Medium	Any
Complexity	Any	Limited	No
Use Case	Transformation & Process	Quick Observation, Automation	Scheduling

Data Storing / Sourcing

1. One Lake
2. Shortcuts
3. Materialized Views
4. Mirrored Sources
(CDC)
5. Connectors



02



Business First - Contexts

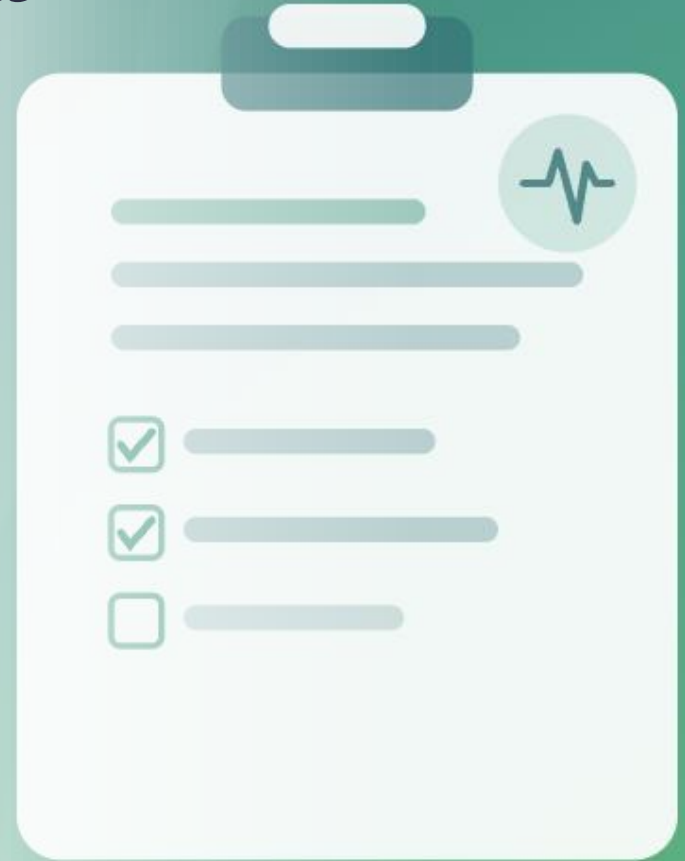


IN THIS SECTION

- ▶ Healthcare Data
- ▶ API data
- ▶ RealTime Data & NRT
- ▶ Operational Automation
- ▶ Complex Logic
- ▶ Data Cleaning

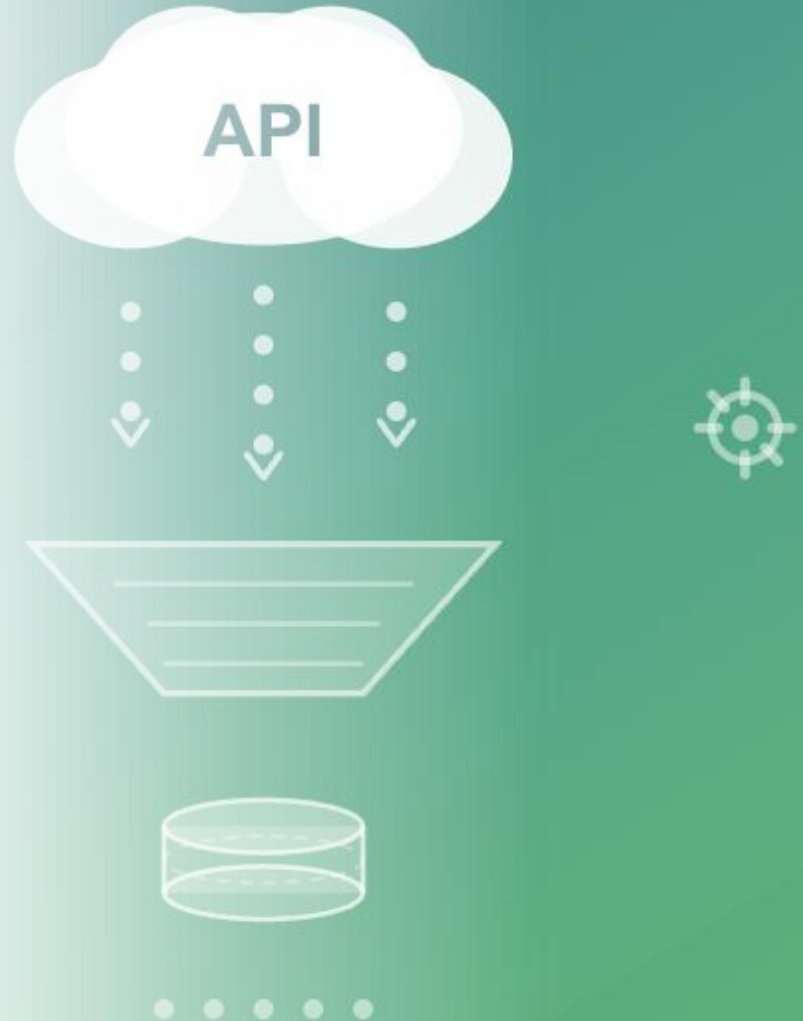
Complex Healthcare Data

- HIPAA Regulations
- HL7 Data Extraction
- FHIR Data Harmonization



API Data Ingestion

- System Integration
- Exploding / Normalizing
- Pagination
- Rate limiting
- Authentication complexity



Real-Time & Near Real-Time Pipelines

-
- Fraud detection
 - Inventory tracking
 - Users' collection / synchronization



Operational Automation

-
- Customer churn detection
 - Lead list Automation
 - Regulatory Reports Generation



Complex Logic Implementation

- Care Gap Identification
- Claims Adjudication
- Lead List Generation



if $x > t$

$\lambda = 0.95$
 $\sqrt{}$

Data Cleaning at Scale

- Users' Matching & Deduplication
- Collecting Users
- Last state of the Order



03



Technical Level in Spark

IN THIS SECTION

- ▶ Idempotence
- ▶ API (explode, text, jsonl)
- ▶ Custom libraries (UDF) + its performance
- ▶ Multithreading read & Folders Processing
- ▶ Delta tables optimization
- ▶ Dedup & cleaning
- ▶ Tuning performance

Idempotence = Same Result

	Idempotent Solution	Non-idempotent Solution
Failure	Rerunning	Duplicates
Incident Recovery	Fast	Complex
Business Metrics	Clear	Vague / Not trustable
Engineering Confidence	Safe by Design	Don't touch it!

API – Processing & Storing

1

How to Store?

Bronze - JSONL.

Silver – array.

Gold – NF3

2

How to Transform?

(1) SELECT Column.Subcolumn

(2) SELECT Column[Index]

(3) Explode / Explode_Outer

3

Logistics

Authentication, Pagination, Rate Limit

UDF functions & Performance

- Is it really slow?

```
1  @udf(StringType())
2  def parse_json(json_str, key):
3      if json_str is None:
4          return None
5      try:
6          data = json.loads(json_str)
7          return str(data.get(key))
8      except (json.JSONDecodeError, TypeError):
9          return None
10
11 # Test it
12 df = spark.createDataFrame([
13     ('{"name": "Alice", "age": 30}',),
14     ('{"name": "Bob", "age": 25}',),
15     ('invalid json',),
16     (None,)
17 ], ["raw_json"])
18
19 df = df.withColumn("name", parse_json("raw_json", lit("name")))
20 display(df)
```

✓ 3 sec - Command executed in 3 sec 135 ms by Iurii Lurchenko on 3/13/2026, 6:55:42 PM

>  Spark jobs (3 of 3 succeeded)  Resources  Log

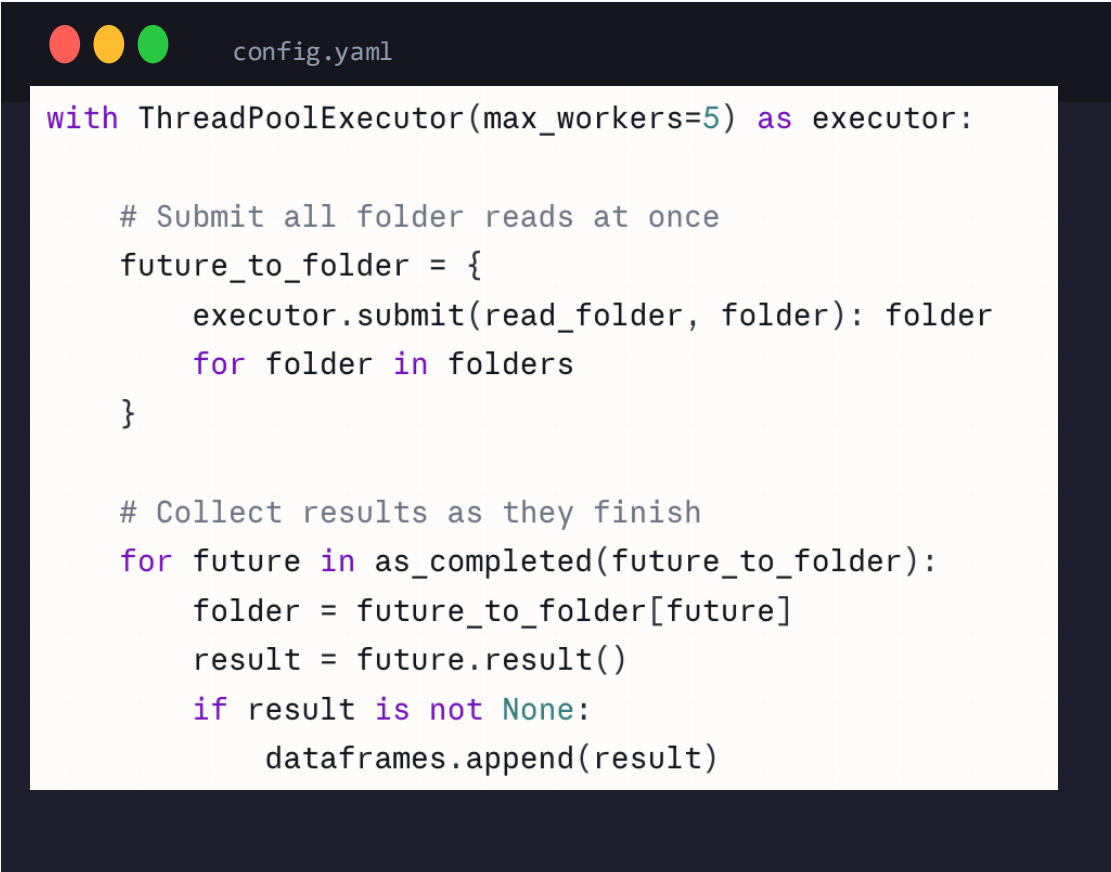
 Table

+ New chart

Table view

	ABC raw_json	ABC name	
1	{"name": "Alic...	Alice	
2	{"name": "Bob...	Bob	
3	invalid json	NULL	
4	NULL	NULL	

Multithreading Read Strategies



```
config.yaml

with ThreadPoolExecutor(max_workers=5) as executor:

    # Submit all folder reads at once
    future_to_folder = {
        executor.submit(read_folder, folder): folder
        for folder in folders
    }

    # Collect results as they finish
    for future in as_completed(future_to_folder):
        folder = future_to_folder[future]
        result = future.result()
        if result is not None:
            dataframes.append(result)
```

- **Reading files one-by-one for unioning by name**

Delta Table Optimization

- **OVERWRITE = DUPLICATE**
- **VACUUM()**
- **OPTIMIZE ... ZORDER ...**

The screenshot displays a data management interface with two main panels. The left panel, titled 'Explorer', shows a hierarchical view of data items. Under 'OneLake', there is a folder 'CoreBronze' which contains a 'Tables' folder. Inside 'Tables', there are four tables: 'claims', 'encounters', 'encounters_nrt', and 'patient_encounters'. The right panel shows a command execution window. It contains a SQL command: `1 %%sql` and `2 VACUUM CoreBronze.claims`. Below the command, it shows the execution status: `[1] ✓ 59 sec - Session ready in 6 sec 996 ms. Command exe`. Below this, it shows the Spark jobs status: `> ⚙ Spark jobs (20 of 20 succeeded) [Resc]`. At the bottom, there is a 'Table view' section with a table showing the path of the 'claims' table.

Table view	
abc path	
1	abfss://9a0...

Deduplication & Data Cleaning

- **row_number()**
- **over (**
- **partition by order_id**
- **order by updated_dt desc**
- **)**

Performance Best Practices

```
python

from pyspark.sql.functions import broadcast, col

# 1. FILTER first - reduce data early
df = orders.filter(col("amount") > 80)

# 2. CACHE - reuse without recomputing
df.cache()

# 3. Broadcast Join - small table, no shuffle
result = df.join(broadcast(categories), "category")

# 4. Avoid Skew - repartition skewed keys
result = result.repartition(8, "category")
```

- **FILTER/WHERE first**
- **CACHE() if needed**
- **Broadcast Join**
- **No Data Skewing**

04



AI & Spark – From Good to Great

IN THIS SECTION

- ▶ Fundamentals of AI in Spark
- ▶ Custom AI functions
- ▶ Spark & ML (high-levelly)

AI Functions

- ai.analyze_sentiment() – detect emotional state
- ai.classify() – categorize according to your labels
- [NEW] ai.embed() – convert text into vector embeddings
- ai.extract() – extract specific types of information
- ai.fix_grammar() – correct spelling, grammar, and punctuation
- ai.generate_response() – use your own custom prompt!
- ai.similarity() – compare meaning with a single text value or with text in another column
- ai.summarize() – summarize input text
- ai.translate() – translate into another language

Source: <https://blog.fabric.microsoft.com/en-US/blog/29826/>

Custom AI Functions in Spark

```
responses = df.ai.generate_response(  
    prompt="""Analyze this phone review and return ONLY a JSON object with three  
fields:  
- "sentiment": one of "Positive", "Negative", or "Neutral"  
- "score": integer 1 (negative) to 3 (positive)  
- "reason": one of these categories ONLY:  
    Positive: "Battery life", "Camera quality", "Display quality", "Performance",  
"Build quality"  
    Negative: "Battery drain", "Poor camera", "Overheating", "Software bugs",  
"Fragile build"  
    Neutral: "NA"  
Review: {review}  
Return ONLY the JSON object, no explanation. """,  
    is_prompt_template=True,  
    output_col="reason_ai",  
    response_format="json_object",  
)|
```

Spark & ML Integration

1. Data preparation
2. Modeling
3. Evaluation
4. Deployment

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline

# Sample data
data = spark.createDataFrame([
    (0, 25, 50000, 0),
    (1, 45, 90000, 1),
    (2, 35, 70000, 1),
    (3, 22, 30000, 0),
    (4, 50, 95000, 1),
    (5, 28, 40000, 0),
], ["id", "age", "income", "bought"])

# Assemble features into a single vector
assembler = VectorAssembler(
    inputCols=["age", "income"],
    outputCol="features"
)

# Model
lr = LogisticRegression(
    featuresCol="features",
    labelCol="bought"
)

# Build pipeline
pipeline = Pipeline(stages=[assembler, lr])

# Train
model = pipeline.fit(data)

# Predict
predictions = model.transform(data)
predictions.select("id", "age", "income", "bought", "prediction").show()
```

sec - Command executed in 43 sec 233 ms by Iurii Iurchenko on 3/13/2026, 7:30:30 PM

05



Fabric's Spark



IN THIS SECTION

- ▶ Modularity (two versions)
- ▶ Environments
- ▶ Key Vault
- ▶ Capacity Consumption & high concurrency

Notebook Modularity

1. DATA PIPELINE
2. %run
3. Notebook.run()

```
1 mssparkutils.notebook.run(  
2   ... "child_notebook",  
3   ... timeout_seconds=300,  
4   ... arguments={"env": "prod", "date": "2026-03-13"}  
5 )
```

[3] ✓ 13 sec - Command executed in 13 sec 579 ms by Iurii Lurchenko on 3/13/2026, 7:12:49 PM

Activity name	Snapshot	Status
0	child_notebook	✓ Succeeded
'SUCCESS'		

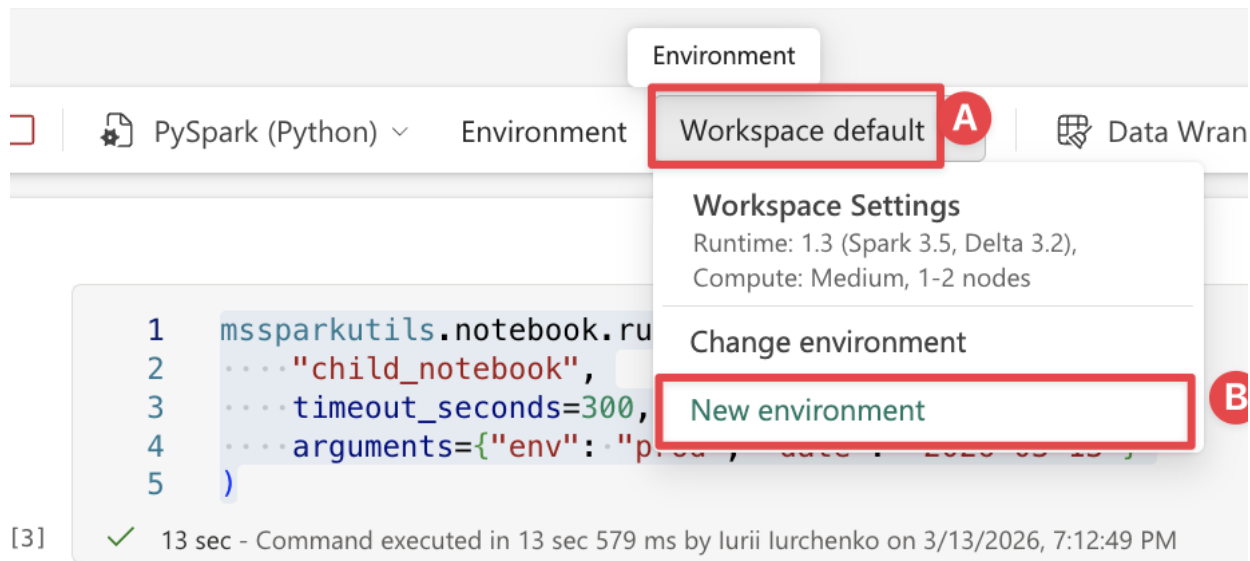
1 %run child_notebook

[4] ✓ 1 sec - Command executed

... Running for dev on 2026-03-13
ExitValue: SUCCESS

Custom Environments

- Spark Limitations (CU)
- Custom Libraries
- Custom Jars (*.whl files)



The screenshot displays the Databricks workspace interface. At the top, there is a toolbar with icons for file operations, a dropdown menu for the current environment (set to 'PySpark (Python)'), and a button labeled 'Environment'. The 'Environment' dropdown menu is open, showing three options: 'Workspace default' (highlighted with a red box), 'Change environment', and 'New environment' (highlighted with a red box). Below the dropdown, a code editor shows a snippet of Python code:

```
1 mssparkutils.notebook.run
2 ... "child_notebook",
3 ... timeout_seconds=300,
4 ... arguments={"env": "pyspark", "date": "2026-03-13"},
5 )
```

 At the bottom of the interface, a status bar indicates that a command was executed successfully in 13 seconds and 579 milliseconds by user 'lurii lurchenko' on 3/13/2026 at 7:12:49 PM.

Environment

PySpark (Python) Environment Workspace default Data Wran

Workspace Settings
Runtime: 1.3 (Spark 3.5, Delta 3.2),
Compute: Medium, 1-2 nodes

Change environment
New environment

```
1 mssparkutils.notebook.run
2 ... "child_notebook",
3 ... timeout_seconds=300,
4 ... arguments={"env": "pyspark", "date": "2026-03-13"},
5 )
```

[3] ✓ 13 sec - Command executed in 13 sec 579 ms by lurii lurchenko on 3/13/2026, 7:12:49 PM

Key Vaults

```
VAULT_URL = "https://my-keyvault.vault.azure.net/"
```

```
def get_secret(name):  
    return notebookutils.credentials.getSecret(VAULT_URL, name)|
```

Home

 **keyvault-test-fabcon**   ...  [How do I troubleshoot issues with this Key Vault?](#)

Key vault

 Delete  Move   Refresh  Open in mobile

 **Overview**

 Activity log

 Access control (IAM)

 Tags

 Diagnose and solve problems

 Access policies

 Resource visualizer

 Events

 **Objects**

 Keys

 Secrets

 Certificates

Essentials

Resource group ([move](#)) : [FabCon](#)

Location : East US

Subscription ([move](#)) : [Azure subscription 1](#)

Subscription ID : 8f309ecc-81ef-4cdd-9e0d-71c6689bdd26

Tags ([edit](#)) : test : supersecret

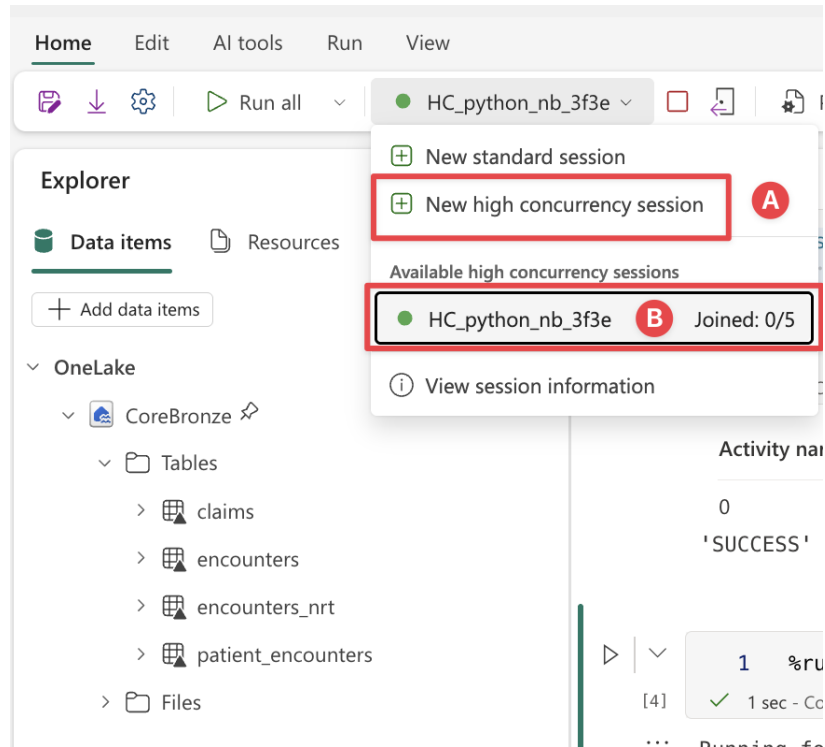
[Get started](#) Properties Monitoring Tools + SDKs Tutorials

Manage k

Our recommendation i
Production and Producti

Capacity Consumption & High Concurrency

- Minimal Spark Cluster = 8 CU
- High Concurrency helps
- Caching - blocks downscale



06



Spark – Decision Making

IN THIS SECTION

- ▶ Spark vs Data Pipelines vs Power Query
- ▶ When Spark related projects fail?
- ▶ Spark SQL vs PySpark vs (PySpark & Spark SQL)
- ▶ Conclusion
- ▶ Questions

Open Floor



Q & A

Ask anything. No question is too basic.

Clarification

"Can you explain X in more detail?"

Application

"How does this apply to our use case?"

Next Steps

"What should we do first on Monday?"